

GlacierFlow

AWS-Native ETL & Data Warehouse Modernization Platform

Glue + Lambda + Athena + Redshift + DMS + Terraform

Every piece of this that can run without a live AWS account, does: real PySpark ETL, real Lambda tests, real migration validation, real Athena partition-pruning measurement, real Terraform validated with zero errors.

Christian Kouadio Kouassi

CONTEXT

The Problem

- A legacy on-prem RDBMS needs to migrate to a modern AWS data platform, cleanly and verifiably.
- No silently lost or duplicated rows along the way, the exact failure mode a migration project gets judged on.
- The target data also has to leave behind a partitioning strategy that actually pays off at query time.

Where the Data Comes From

- Synthetic legacy enterprise data: 5,000 customers, 800 products, 22,000 orders, 66,031 order line items.
- Realistic legacy-system quirks: SCREAMING_SNAKE_CASE columns, ~4% missing signup dates.
- The kind of light schema drift a real migration source actually has, not a clean synthetic dataset.

HYPOTHESIS

What We're Testing For

- Whether a migration's correctness can be verified independently of the transformation that produced it, the same principle DMS's own validation is built on.
- Whether partitioning by the dimension a dashboard actually filters on (month) produces a real speedup.
- Both held up: 3 of 3 tables passed row-count and checksum validation, and partition pruning measured a real 3.9x speedup.

Stack & Verification

Layer	Tool	Verification
Source migration	AWS DMS (documented)	Row-count + checksum reconciliation, real PASS
Transformation	AWS Glue (PySpark)	Real, executing bronze/silver/gold ETL
Trigger + quality gate	AWS Lambda	Real handler, moto-mocked tests, 4/4 passing
Serverless SQL	Amazon Athena (documented)	Partition pruning measured for real via DuckDB
Warehouse	Amazon Redshift (documented)	Schema matches the same gold-layer Parquet
IaC	Terraform	15 resources, 0 validation errors

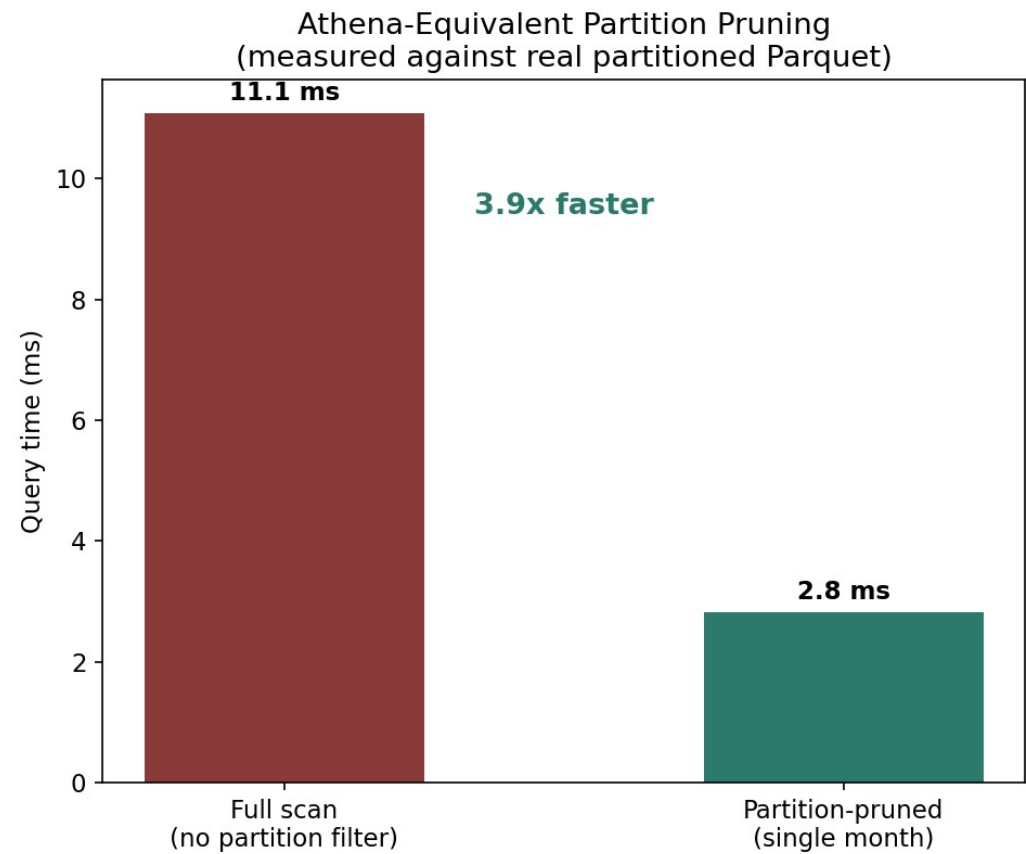
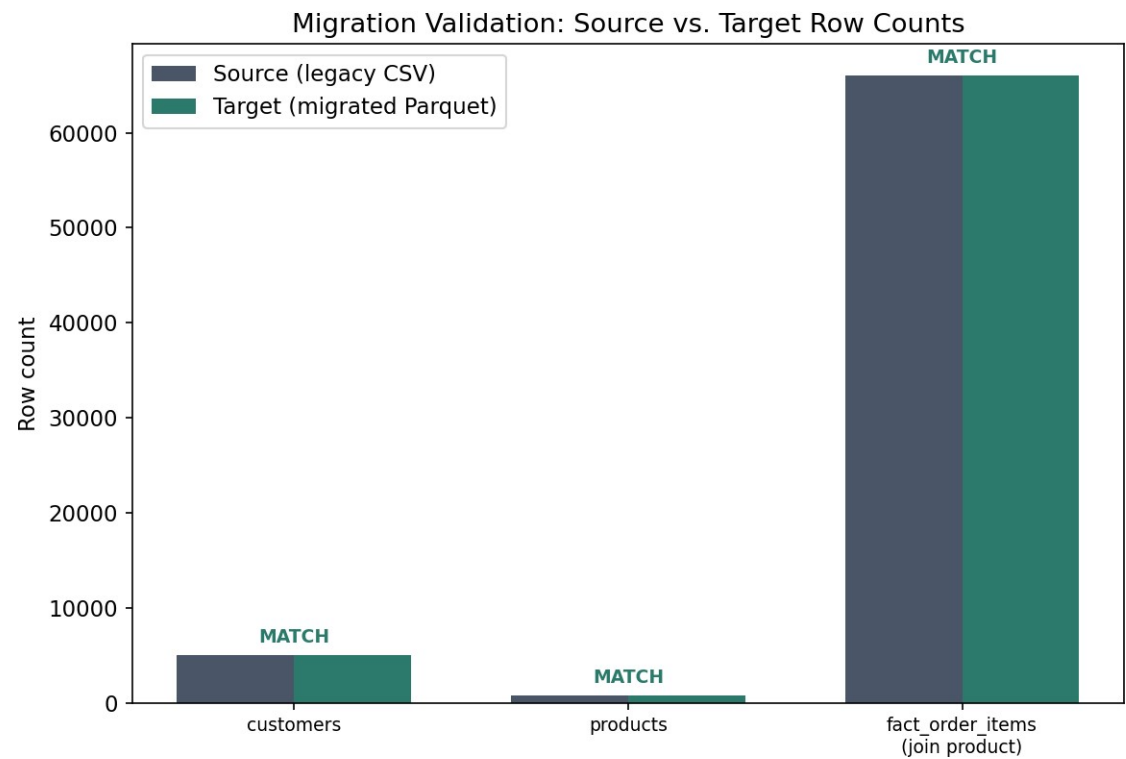
SCOPE

The Task

- Migrate legacy on-prem data through a bronze/silver/gold Glue pipeline.
- Validate the migration independently of the transformation that produced it, the reconciliation discipline this repo is built around.
- Partition the gold layer for real query-time performance, verified against the actual Parquet output.
- Provision the whole thing as Terraform, and gate a new file's entry into the pipeline with a real Lambda quality check.

KPI SNAPSHOT

Results



3 of 3 tables validated (row count + checksum). \$36.3M total revenue computed. 4.2% missing signup dates caught, not dropped. Estimated Glue cost: \$0.18/run, ~\$5.28/month at daily cadence.